

Lab 3. Protocolo de Enrutamiento

José Mérida - 201105 & Angel Esquit - 23221
Roberto Barreda - 23354 & Javier España 23361
Anggelie Velasquez - 221181 & Mia Fuentes - 23775

6 de agosto de 2026

Introducción

El punto de partida es el formato de LSA que ya da el enunciado del laboratorio, así que diseñamos todo lo demás alrededor de eso en vez de inventar un esquema nuevo: HELLO se mantiene mínimo (solo confirma que el vecino sigue vivo), el costo de enlace se asume 1 por defecto por simplicidad, y el manejo de ciclos se resuelve con número de secuencia por origen. Para el plano de datos seguimos el formato `{nodo_origen, nodo_destino, mensaje}` del enunciado tal cual, y adentro de `mensaje` va el JSON de aplicación que ya usamos en el Lab 2, sin modificarlo, para no tener que rehacer esa parte. En general la idea fue mantener el protocolo lo más simple posible y solo definir lo estrictamente necesario para que las tres parejas puedan interoperar.

1. Paquete LSA

```
{
  "type": "LSA",
  "origin": self.self_ip,
  "seq": self.seq,
  "links": links,
  "from": self.self_ip
}
```

- `origin` = quién generó el LSA originalmente; `from` = quién me lo acaba de reenviar (el salto inmediato anterior).
- `links`: lista de vecinos directos de `origin` con su costo, ej. `[{"to": "R2", "cost": 1}, ...]`.

2. Paquete HELLO

```
{
  "type": "HELLO",
  "from": self.self_ip
}
```

Cada nodo lo manda periódicamente a sus vecinos directos para descubrirlos / saber que el enlace sigue vivo. Si un vecino deja de mandar HELLO por un tiempo, se asume caído y se regenera el LSA propio con ese link quitado.

Por defecto, todos los enlaces tienen costo 1 (no se manda costo por HELLO). Como extensión opcional, se puede agregar un campo `cost` al HELLO para declarar el costo del enlace hacia ese

vecino:

```
{
  "type": "HELLO",
  "from": self.self_ip,
  "cost": 1
}
```

Si un nodo no manda `cost`, se asume 1.

3. Reenvío periódico del LSA

Además de reenviarse al cambiar el estado de un enlace, cada nodo re-emite su propio LSA periódicamente (con `seq` incrementado), haya cambiado algo o no. Esto asegura que un nodo que se une tarde a la red, o que se recupera de una partición, termine recibiendo el estado completo de la topología sin depender de que justo en ese momento ocurra un cambio de enlace en otro nodo.

4. Manejo de ciclos en el flooding

La topología tiene ciclos físicos, así que sin esto el LSA daría vueltas indefinidamente.

- Cada nodo guarda el último `seq` visto por cada `origin`.
- Si llega un LSA con (`origin`, `seq`) ya visto o más viejo, se descarta (no se reenvía).
- Si el `seq` es más nuevo, se guarda, se recalcula ruta (Dijkstra) y se reenvía a todos los vecinos *menos* al que lo mandó (usando `from`).

5. Envío de datos: mensaje cliente/servidor

```
{nodo_origen, nodo_destino, mensaje}
```

Los routers intermedios solo leen `nodo_destino`, consultan su `nodo_tabla_enrutamiento.csv` y reenvían, no tocan `mensaje`.

`nodo_origen` / `nodo_destino` son los IDs de topología del ATM y del servidor bancario (los mismos nombres que aparecen como `origin` en los LSA). `mensaje` es directamente el JSON de aplicación del Lab 2 (login/withdraw/error/logout), sin modificar su estructura interna.

```
{
  "nodo_origen": "ATM1",
  "nodo_destino": "BANK1",
  "mensaje": {"action": "withdraw", "data": {"amount": 100}}
}
```

La respuesta simplemente invierte `nodo_origen` / `nodo_destino`.

6. Formato de mensaje (aplicación, Lab 2)

Dentro de `mensaje` va el JSON de aplicación tal como se usa en el Lab 2: `{.action": ..., "data": {...}}`. Las acciones soportadas son `login`, `withdraw`, `logout` y sus respectivas respuestas (`login_ok/login_denied`, `withdraw_ok/withdraw_denied`, `logout_ok`), más `error` para cualquier caso inválido.

```
// ATM -> Banco: login
{
  "nodo_origen": "ATM1", "nodo_destino": "BANK1",
  "mensaje": {"action": "login", "data": {"card": "4111111111111111", "pin":
    "1234"}}}
}

// Banco -> ATM: login aceptado
{
  "nodo_origen": "BANK1", "nodo_destino": "ATM1",
  "mensaje": {"action": "login_ok", "data": {"message": "Authentication successful
    "}}}
}

// ATM -> Banco: retiro
{
  "nodo_origen": "ATM1", "nodo_destino": "BANK1",
  "mensaje": {"action": "withdraw", "data": {"amount": 100}}}
}

// Banco -> ATM: retiro exitoso
{
  "nodo_origen": "BANK1", "nodo_destino": "ATM1",
  "mensaje": {"action": "withdraw_ok", "data": {"amount": 100, "balance": 400.00}}}
}

// Banco -> ATM: error (p. ej. fondos insuficientes, no autenticado, JSON invalido)
{
  "nodo_origen": "BANK1", "nodo_destino": "ATM1",
  "mensaje": {"action": "error", "data": {"message": "Insufficient funds"}}}
}

// ATM -> Banco: logout
{
  "nodo_origen": "ATM1", "nodo_destino": "BANK1",
  "mensaje": {"action": "logout", "data": {}}}
}
```

Este formato de mensaje es el mismo del Lab 2 sin cambios; lo único nuevo es que ahora viaja envuelto en `nodo_origen/nodo_destino` para poder atravesar los routers.

7. Tabla de enrutamiento (`nodo_tabla_enrutamiento.csv`)

Cada nodo escribe su tabla de ruteo, calculada con Dijkstra sobre el grafo armado a partir de los LSA, en un archivo CSV con una fila por destino alcanzable:

```
destino,siguiente_salto,costo,ip,puerto
R2,R2,1,127.0.0.1,5002
R3,R2,4,127.0.0.1,5002
BANK1,R3,6,127.0.0.1,5010
```

- **destino:** nodo final al que se quiere llegar (router, ATM o banco).
- **siguiente_salto:** vecino directo al que hay que reenviar para llegar a **destino** por la ruta más corta.
- **costo:** costo total acumulado de la ruta.

- `ip/puerto`: dirección del `siguiente_salto`, para abrir el socket.

Cuando un router recibe `{nodo_origen, nodo_destino, mensaje}`, busca `nodo_destino` en este CSV y reenvía el paquete completo (sin modificarlo) a la `ip/puerto` de esa fila. El archivo se regenera cada vez que cambia la ruta más corta a algún destino (es decir, después de procesar un LSA nuevo).